

WEMOVE 项目沟通记录

目录

一、模块对接沟通记录	2
1 王鹤（A）与冷雨（B）：开发依赖	2
2 冷雨（B）、夏俊杰（C）、徐熠潘（E）：项目目录重构	2
3 冷雨（B）与夏俊杰（C）：商品数据获取	4
4 冷雨（B）与夏俊杰（C）：购物车商品变化	5
5 王鹤（A）与杨振煜（D）：经销资格	6
6 冷雨（B）与杨振煜（D）：经销商品目录	7
7 冷雨（B）与徐熠潘（E）：商品图片文件	8
8 杨振煜（D）与徐熠潘（E）：经销资料下载	8
9 夏俊杰（C）与戴靖桓（F）：工单订单关联	9
10 徐熠潘（E）与 GPT 的对话记录	10

i 内容说明

本记录仅为便于展示，采用“对话过程 + 形成的总结”的记录形式。内容根据群聊、私聊、线下讨论等多个沟通渠道中的有效信息提取、整理而成，不等同于任一单一渠道的完整原始记录。

一、模块对接沟通记录

1 王鹤（A）与冷雨（B）：开发依赖

讨论：B 是否需要等待 A 开发完成

冷雨（B）

我这边商品和库存模块是不是要等你身份这部分开发完才能开始？感觉后面接口都要判断用户身份。

王鹤（A）

需要等我这边身份与权限模块开发完毕，接口稳定后你再开始商品和库存的正式开发。

冷雨（B）

明白，那我先等你的身份模块完成，再开始商品和库存这边的开发？

王鹤（A）

对。等身份和权限接口稳定后，我们再一起确认登录、管理员权限这些调用方式。

冷雨（B）

好的。

形成的正式结论

经本轮沟通确认，商品与库存模块的正式开发以前置身份与权限模块为依赖。A 完成身份、权限相关实现并使接口达到可联调状态后，B 再开展商品与库存的正式开发；登录态、管理员权限等具体调用方式由 A、B 在接口稳定后共同确认，后续联调以双方确认的接口行为为准。

2 冷雨（B）、夏俊杰（C）、徐熠潘（E）：项目目录重构

讨论：项目仍沿用 A 成员目录

夏俊杰（C）

我准备接商品模块开始做交易这部分了，不过现在整个工程是不是还放在 partA 下面？

冷雨（B）

对，我当时是在 A 已经写好的项目上直接接着开发，所以 B 的商品和库存也放进去了。

夏俊杰 (C) 

现在已经不只是 A 的代码了, 如果后面 C、D、E、F 都继续往这里放, 目录看起来会比较乱。

 徐熠潘 (E)

我也觉得需要整理。成员字母可以表示分工, 但项目代码不应该继续按成员目录来分。

 冷雨 (B)

可以, 我当时主要是为了直接接现有工程, 没有再改外层结构。

 徐熠潘 (E)

那我先把项目代码整理到统一的 `project-implementation` 下面, 前后端按整个项目来放。

讨论: 后端包结构也存在类似问题

夏俊杰 (C) 

外层目录改完以后, 后端里面还有一个问题。现在商品代码是在 `identity` 下面, 我感觉商品和身份应该是同一级。

 冷雨 (B)

这个也是因为我直接在 A 的后端工程里往下加, 所以当时就接在现有包下面了。

 徐熠潘 (E)

既然外层已经统一成项目结构, Java 包这里也最好一起理顺。

夏俊杰 (C) 

那我来改后端这部分。最外层统一成 `wemove`, 下面再放 `identity`、`catalog`、`commerce` 这些模块。

 冷雨 (B)

可以, 只要商品原来的功能不受影响就行。

夏俊杰 (C) 

我只整理目录和包结构, 改完把现有测试再跑一遍。

 徐熠潘 (E)

行，后面其他模块也按这个结构继续接。

✓ 形成的正式结论

1. 项目代码不再以成员字母或 `partA`、`partB` 等成员目录作为业务组织边界。徐熠潘负责将前后端代码统一归入 `project-implementation`，使项目结构能够反映工程职责和业务范围。
2. 夏俊杰负责调整后端 Java 包结构，将最外层统一为 `wemove`，并使 `identity`、`catalog`、`commerce` 等业务模块保持平级，避免商品或交易代码继续嵌套在身份模块之下。
3. 后续新增模块直接接入统一工程结构，不再按成员目录扩展。目录与包结构调整完成后，继续使用现有测试验证既有功能不受迁移影响，并以该结构作为后续模块协作的共同基础。

3 冷雨 (B) 与夏俊杰 (C)：商品数据获取

讨论：交易模块如何获取商品数据

夏俊杰 (C) 

我这边购物车和结算都要用商品价格、库存和上架状态。我是直接查你商品模块的表，还是你这边给我接口？

 冷雨 (B)

直接从我这边拿吧。我商品模块里这些数据都有，你如果直接查表，后面我这里字段或者实现调整了，你那边也得跟着改。

夏俊杰 (C) 

那你给我一个统一的方法？我传商品 ID 和数量，你把当前价格、库存、是不是上架这些返回给我。

 冷雨 (B)

可以。购物车和结算需要的数据我整理成一个返回对象，你直接用这个就行。

夏俊杰 (C) 

那我 C 这边就不自己查商品表了，购物车和订单统一从 B 这里取。

 冷雨 (B)

对，这样商品这边后面调整也比较处理好。

形成的正式结论

C 不直接读取 B 的商品和库存数据表，B 作为商品数据的唯一业务来源，提供统一的商品数据读取能力。该能力至少根据商品 ID 和数量返回交易模块需要的当前价格、库存和商品状态等信息；购物车与订单流程统一通过该能力获取数据，从而隔离数据表结构变化并保持模块边界清晰。

4 冷雨（B）与夏俊杰（C）：购物车商品变化

讨论：商品加入购物车后发生改价或下架

夏俊杰（C）

还有个情况，商品已经加进购物车以后，如果你那边后来改价格或者把商品下架了，我购物车里还是保留原来的信息吗？

冷雨（B）

当前商品状态还是以我这里最新的数据为准吧。尤其是下架和库存，不能因为之前加过购物车就一直可以买。

夏俊杰（C）

那我打开购物车的时候重新取一次商品信息？

冷雨（B）

对，价格、库存和状态都重新拿。购物车里可以保留之前的数据做比较，但最终还是看当前商品信息。

夏俊杰（C）

那如果价格变了，我就显示新的价格；如果下架或者库存不够，就提示这个商品现在不能结算。

冷雨（B）

可以。

夏俊杰（C）

结算的时候我再检查一次，防止用户打开购物车以后商品又发生变化。

冷雨（B）

嗯，这样稳一点。

形成的正式结论

购物车中保存的加入时信息仅作为过程数据，不作为后续交易判断的最终依据。展示购物车时重新读取 B 模块提供的当前价格、库存和商品状态；如果价格发生变化，应更新页面提示，如果商品已下架或库存不足，应明确标记为不可结算。用户进入结算流程时还需再次执行校验，以覆盖打开购物车后商品状态继续变化的情况，最终结算结果以接近提交时获取的最新数据为准。

5 王鹤（A）与杨振煜（D）：经销资格

讨论：经销商是否需要新增用户角色

杨振煜（D）

我这边经销合作审核通过以后，这个人算不算一个新的用户角色？要不要在 A 那边再加一个 DEALER？

王鹤（A）

我觉得先不要加。现在用户这边主要就是普通用户和管理员，经销商更像是普通用户额外有了一个合作资格。

杨振煜（D）

那我判断能不能进经销专区的时候，除了登录状态，还得查我这里的合作状态？

王鹤（A）

对。A 这边负责告诉你当前登录的是谁、账号是不是正常，你再判断这个用户有没有有效的经销资格。

杨振煜（D）

这样以后合作暂停了，也不用去改用户角色。

王鹤（A）

对，不然账号角色和合作状态容易出现对不上的情况。

形成的正式结论

用户的基础身份、登录状态和账号有效性仍由 A 统一管理，不在身份模块中额外增加 DEALER 基础角色。经销资格属于随企业合作状态变化的业务资格，由 D 负责维护和判断；访问经销专区时同时满足有效登录状态与当前有效经销资格，合作暂停或终止时无需修改基础用户角色即可立即反映权限变化。

6 冷雨（B）与杨振煜（D）：经销商品目录

讨论：经销专区的商品从哪里获取

杨振煜（D）

我经销专区也要展示商品，这部分是不是直接用你现在公开商品的接口？

冷雨（B）

普通商品的信息可以共用，但你那边还需要经销参考价、最低询价数量这些，公开商品接口现在没有。

杨振煜（D）

那你给我一份经销模块用的商品数据吧？商品名、SKU、经销参考价、最低数量这些我直接拿。

冷雨（B）

可以。商品本身还是我这里维护，我把你需要的字段提供出来。

杨振煜（D）

经销资格要不要你这边一起判断？

冷雨（B）

资格还是你判断比较合适。我这里只负责商品数据，不知道这个用户现在是不是有效经销商。

杨振煜（D）

行，那我先检查资格，通过以后再取你这边的经销商品信息。

形成的正式结论

B 继续作为商品信息、SKU、经销参考价、最低询价数量等数据的维护方，并通过约定的读取能力向 D 提供经销场景所需字段。D 负责先检查当前用户的经销资格，再组织经销专区的展示和业务处理；经销模块不复制商品主数据，也不在 B 中重复实现资格判断，以保持数据归属和业务职责明确。

7 冷雨（B）与徐熠潘（E）：商品图片文件

讨论：商品图片由哪个模块负责

冷雨（B）

我商品后台要支持上传商品图片，这部分文件上传是不是也放我商品模块里面？

徐熠潘（E）

我这边本来就要做文章图片、下载资料和媒体文件，如果商品再自己做一套上传，后面会有两套文件逻辑。

冷雨（B）

那商品这边怎么保存图片？总得跟商品关联起来。

徐熠潘（E）

你保存文件的 ID 或引用就行，真正的上传、文件保存和下载放我这里。

冷雨（B）

那我商品详情拿到这个文件 ID，再通过你这边拿图片？

徐熠潘（E）

对。这样以后文件存放方式改了，也不用改商品表。

冷雨（B）

可以，那我商品这里只管图片跟哪个商品关联。

形成的正式结论

商品模块不单独实现文件存储能力。E 统一负责商品图片、文章图片及其他媒体文件的上传、保存和访问，B 只在商品数据中保存文件 ID 或其他稳定引用，并维护商品与图片之间的关联。商品详情需要展示图片时通过 E 获取文件，后续更换存储方式或文件访问实现时无需改动商品模块的数据职责。

8 杨振煜（D）与徐熠潘（E）：经销资料下载

讨论：经销专属资料如何限制访问

杨振煜（D）

我这边经销商审核通过以后，可以看到一些专属资料。这些文件是不是直接放到页面上给下载？

徐熠潘 (E)

公开资料可以，但经销专属文件最好不要直接给一个一直有效的下载地址，不然链接转出去以后别人也可能打开。

杨振煜 (D)

那每次下载的时候都要重新检查经销资格？

徐熠潘 (E)

对。你先判断这个用户现在还有没有资格，通过以后再让他访问对应文件。

杨振煜 (D)

如果这个人之前下载过，但后来合作状态被暂停了呢？

徐熠潘 (E)

那之后再请求就不给了。以前有权限不代表以后一直有。

杨振煜 (D)

明白，那专属资料入口我来控制，文件本身还是你管理。

形成的正式结论

E 负责经销资料文件本身以及文件访问能力，D 负责维护并判断用户当前是否具有有效经销资格。用户每次请求经销专属资料时都应先由 D 重新校验资格，再由 E 提供对应文件；资格失效后，后续请求立即拒绝，即使用户此前曾经下载过，也不继续沿用历史权限。访问入口不采用长期公开的固定下载地址，以维持经销资料的访问边界。

9 夏俊杰 (C) 与戴靖桓 (F)：工单订单关联

讨论：工单如何关联订单

戴靖桓 (F)

我这边用户提交咨询或者售后工单的时候，想让他可以选一个相关订单。这个订单信息我是直接从订单表查吗？

夏俊杰 (C)

最好从我这里拿。订单有用户归属，如果直接按订单 ID 查，还得注意不能让用户看到别人的订单。

戴靖桓 (F)

那我传当前用户和订单 ID，你帮我判断是不是他的？

夏俊杰 (C)

可以。我给你必要的订单信息就行，比如订单号、状态这些。

戴靖桓 (F)

工单里要不要把整份订单信息复制下来？

夏俊杰 (C)

没必要。保存订单关联和你工单真正需要的少量信息就够了，订单本身还是我这里维护。

戴靖桓 (F)

那后台客服看工单的时候，再根据这个关联去看订单现在是什么状态。

夏俊杰 (C)

对，这样不会出现两份订单数据对不上。

形成的正式结论

F 的工单可以关联 C 的订单，但工单模块不复制完整订单业务数据。用户选择订单时，由 C 根据当前用户身份和订单 ID 判断订单归属，并向 F 提供订单号、状态等客服处理所需的必要只读信息；F 保存订单关联和工单业务需要的最小信息，订单主体及其最新状态仍由 C 维护，后台查看工单时根据关联获取当前订单信息，避免形成两份不一致的订单数据。

10 徐熠潘 (E) 与 GPT 的对话记录

一、抽象问题讨论：从“好网站”到可协作的系统

1. 完整且优秀的网站应具备什么？

徐熠潘 (E)

一个完整且优秀的网站应该具备哪些特质？需要有哪些组成部分？各组成部分需要满足什么要求？

GPT

完整不等于页面多，优秀也不等于视觉精致。网站应能稳定地帮助目标用户完成目标，并可维护、可演进。

可将网站类比为一家实体店：目标与需求决定服务谁、卖什么；信息架构与导航相当于货架布局；页面与交互决定逛店体验；后端、数据库与接口

对应库存、收银和供应链；安全、性能与运维则提供门锁、通道和日常保障。

核心质量包括：

- **有效性**：支撑注册、查询、购买、提交等目标任务的完整闭环；
- **易用性**：用户能理解入口、操作结果及下一步；
- **可靠性与性能**：弱网、空数据、错误操作等情形仍有明确反馈和恢复路径；
- **安全与可信**：保护账户、隐私和业务数据，清楚说明身份、价格和规则；
- **可访问性**：兼顾键盘操作、可读文字、适当色彩对比等；
- **可维护性**：结构清晰、组件可复用、文档与测试可追踪。

需求应写成可验收的陈述。例如，与其说“网站要方便”，不如约定“新用户应能在 3 分钟内完成注册并找到课程报名入口”。前者是感觉，后者可以被测试和复核。

2. 网页的底层实现包含哪些部分？

徐熠潘（E）



网页的底层实现包含哪些部分？比如前端、后端、数据库，除此之外还有什么？

GPT

前端、后端和数据库是核心三层；面向真实上线的网站还会配置若干支撑层。一条典型访问链路可以抽象为：

浏览器 → DNS / CDN → 负载均衡或网关

→ 前后端服务 → 缓存、数据库、文件存储、消息队列

除三层外，常见的能力包括：

- **Web 服务器或反向代理**：接收 HTTP 请求、托管静态文件、转发服务；
- **身份认证与权限**：确认“你是谁”，并决定“你能做什么”；
- **文件或对象存储**：保存图片、视频、附件等非结构化文件；
- **缓存**：保存高频访问数据以减少数据库压力；
- **消息队列**：把邮件、转码、批处理等耗时工作改为异步完成；
- **搜索、日志、监控与告警**：分别服务检索能力和运行问题定位；
- **部署、网络与安全设施**：保证公网访问、扩容、防护和恢复。

这些组件不是必选清单，而是应随业务规模和风险增加。小型项目先确保核心业务链路清楚，再引入额外基础设施。

3. 不上线的课程网站可省略什么？

徐熠潘 (E)

我们的网站不需要上线，从这个角度出发，有哪些组成部分是我们不需要的？

GPT

如果只在本地演示，不必建设生产环境的承载与运维体系。通常可省略域名与 DNS、云服务器、HTTPS 证书、CDN、负载均衡、容器编排、API 网关、Redis 缓存、消息队列、完整监控告警、CI/CD、WAF/DDoS 防护、高可用/灾备和 SEO。

但“本地演示”并不意味着可以省略软件质量。仍应保留或按需求选择：

- 前端界面、响应式布局与交互；
- 需要真实登录、角色或增删改查时所需的本地后端与数据库；
- 基础权限、输入校验、异常提示和防重复提交；
- 可复现的启动说明、Git 协作记录及核心测试。

推荐按功能复杂度选择最小架构：

项目需要	建议结构
纯展示或页面复刻	前端 + 静态资源 / JSON
筛选、收藏、主题等单机交互	前端 + localStorage
模拟登录、商品详情、购物车	前端 + Mock 数据或本地 JSON
真实注册、权限、后台和持久化 CRUD	前端 + 本地后端 + MySQL / SQLite

localStorage 是浏览器本地存储，适合演示收藏、购物车或设置；它不能模拟多用户共享数据，清理浏览器数据后也可能丢失。

4. 网站设计的一般流程是什么？

徐熠潘 (E)

网站设计需要遵循的一般流程是什么？

GPT

常用路径是先定义问题和用户，再组织内容与任务，最后设计、验证并交付：

调研与定义 → 信息架构 → 交互设计 → 视觉设计
→ 原型验证 → 开发交付 → 测试迭代

具体而言：

1. 明确目标、用户角色、功能范围和约束；
2. 通过用户画像、用户故事与任务流程，把抽象功能转成可使用的场景；

3. 建立信息架构（栏目、页面和导航的组织关系），先画网站地图；
 4. 设计用户流程、表单规则和默认/加载/空白/异常/成功等状态；
 5. 先画低保真线框图，确认布局、信息层级与动线；
 6. 制定颜色、字体、间距、组件状态等设计规范，再完成高保真稿；
 7. 用可点击原型请真实用户完成任务，观察卡点；
 8. 向开发交付页面、资源、响应式和交互规则；
 9. 从视觉、功能、设备适配和异常处理四方面验收，再循环迭代。
- 这里要区分：**用户路径**关注用户为达成目标而做出的决策；**页面路径**关注页面之间怎样跳转。两者共同决定体验，不能只画页面而忽略用户为什么会放弃操作。

5. 信息架构和复杂业务逻辑有没有成熟表达方式？

徐熠潘（E） 

建立信息架构、梳理任务逻辑等工作在软件工程中有成熟的工作流？从业者一般用什么来表示复杂业务逻辑？任务流程是否有专门的绘图规则？

 GPT

有。中小型网站可采用“从人和业务逐步落到页面、接口和数据”的工作流：

角色与目标 → 用户故事 / 用例 → 任务与业务流程
→ 数据与页面原型 → 接口契约

不同问题使用不同表示法，而不是让一张流程图承担全部含义：

要说明的问题	推荐产物	说明
谁在使用、权限边界是什么	用户画像、角色表、权限矩阵	将“身份”与“可执行操作”明确对应。
用户为何需要功能	用户故事、用例	用“作为……我希望……以便……”表达价值。
用户怎样完成目标	用户流程图 / Task Flow	关注操作、决策与页面跳转。
多角色业务怎样交接	BPMN、活动图、泳道流程图	适合审核、订单、售后等流程。
模块怎样按时间协作	UML 时序图	展示前端、后端、数据库等消息往返。
对象有哪些生命周期	状态图 / 状态机	规定订单、报名等状态的合法转换。
多条件如何得出结果	决策表、决策树	使资格校验等复杂规则避免变成“意大利面条图”。
数据对象如何关联	ER 图	说明实体、主键、关系和基数。

普通用户流程图可采用：圆角矩形表示开始/结束，矩形表示页面或操作，菱形表示判断，箭头表示方向；多主体时用泳道区分用户、管理员和系统。BPMN（业务流程模型与标记法）则是更严格的标准：圆形表示事件、圆角矩形表示任务、菱形表示网关，泳道明确责任归属。

流程与规则需分开表达：

业务流程 = 步骤顺序

业务规则 = 每一步允许或禁止的条件

例如，报名过程适合用流程图；“已登录、课程有名额、未重复报名、时间有效”等资格组合则更适合决策表。

6. 有哪些便捷的绘图工具？

徐熠潘（E）

上述流程图等图样，是否有现成的网站或工具能便捷生成？

GPT

可按图的类型选择工具：

产物	推荐工具	使用定位
网站地图、用户流程、ER 图、UML 图	diagrams.net (draw.io)	免费、通用、支持导出 PNG/SVG/PDF；只选一个工具时优先选它。
用户画像、用户旅程、需求讨论白板	FigJam	适合多人同步讨论和早期发散。
页面原型与高保真界面	Figma	可制作组件、设计稿和可点击原型。
随 Markdown / 代码版本化维护的图	Mermaid Live Editor	通过文本生成流程图、时序图、ER 图、状态图和甘特图。
严格的审批流或跨角色业务流程	bpmn.io	专门建模 BPMN 2.0 流程。

推荐协作方式是：用 FigJam 讨论需求和用户旅程，用 Figma 产出界面及交互原型，用 diagrams.net 输出正式图，用 Mermaid 维护随代码而变化的技术图。对于课程项目，至少应具备网站地图、核心用户流程、ER 图和角色权限表；有审核或订单逻辑时再补 BPMN 图或状态图。

7. 如何以接口规范兼顾组件自由度与稳定组合？

徐熠潘 (E)

在软件工程实践中，如何确定接口规范，才能保证各组件既有设计自由度，又能稳定组合？

GPT

关键不是接口写得越细越好，而是建立**稳定、最小、可验证、可演进的接口契约 (contract)**。组件依赖契约，而非依赖彼此的内部实现：

$$\text{客户端 } C \text{ 能稳定使用服务 } P \iff P \models K$$

其中， K 是契约， $P \models K$ 表示服务实现满足该契约。只要替换后的实现仍满足 K ，客户端就不必随内部技术或数据库结构变化而改动。
一个完整接口不只是 URL，而至少需要约定五层内容：

层次	应约定的内容
语法	地址、HTTP 方法、字段名、数据格式
数据	类型、必填性、范围、默认值
语义	该操作在业务上代表什么
行为	成功、失败、重复请求、冲突时的结果
非功能约束	权限、分页、幂等、性能、限流等

建议采用 API First（接口优先）：先明确业务规则并设计 OpenAPI 规范，前后端共同评审；前端可基于 Mock 数据并行开发，后端按规范实现，最后以自动化测试验证契约。

兼容性不是“从此不能改”，而是保证变化可预期。一般可新增接口或响应中的可选字段；删除字段、改变字段类型/语义、把可选改为必填、改变错误码含义，都属于不兼容变化。若旧客户端可发送的请求集合为 R_{old} ，新版本理想上应满足：

$$R_{old} \subseteq R_{new}$$

但字段可接收还不够，响应语义、权限、状态码和副作用也要保持承诺；因此真正重要的是**行为兼容**。无法兼容的重大改动应采用版本化接口，例如 `/api/v1/...` 与 `/api/v2/...`。

验证至少应包含接口单元测试、集成测试和契约测试。对于审核、订单等复杂对象，接口文档还应与状态图、决策表、时序图配套，分别回答“状态能否转换”“条件如何判断”“模块如何协作”。

二、细节问题讨论：将 WEMOVE 的业务落为文档与技术方案

1. 网站重构面向哪些用户？

徐熠潘（E）

阅读《网站重构需求.docx》后，需要重构的网站的目标用户群体是什么？

GPT

目标用户是成年人，而非儿童本身，主要包括家长与成年消费者、礼品购买者、零售商/批发商/进口商及其采购人员、教育或活动机构采购人员，以及已合作或潜在经销商。
尽管产品服务儿童运动、协调能力和亲子互动，账户注册、交易及个人信息收集均应面向成年人：不创建儿童账户，也不主动收集儿童个人信息。

2. 用户需要完成哪些任务？

徐熠潘 (E)



用户要完成什么？例如浏览、查询、报名、购买、发布内容。

GPT

WEMOVE 的用户任务可归为五类：

- **浏览与查询：**按年龄、玩法、场景筛选产品；搜索产品、文章、FAQ 和下载资料；查找经销商。
- **购买与售后：**收藏、加购、结算、查订单和申请退换；未开通零售时应转为“在哪里购买”或联系咨询。
- **商务合作：**潜在经销商申请合作；获批经销商查看专属目录、价格、库存和交期，询价、批量下单、下载私有资料，管理企业与成员信息。
- **内容与支持：**阅读玩法指南和安全信息，下载说明书/证书，提交联系或售后表单，订阅邮件。
- **运营管理：**后台人员维护商品、价格、订单、经销商、文章、Banner、FAQ、SEO 与报表。

前台用户的主链路可概括为“了解 → 比较/咨询 → 购买或合作”，后台的主链路为“维护 → 运营 → 分析”。

3. 项目的最终目标是什么？

徐熠潘 (E)



项目要达成什么？例如展示品牌、完成网站复刻、提高报名效率。

GPT

核心目标不是机械复刻旧页面，而是将静态品牌站升级为可持续运营的业务门户。目标包括：展示品牌、产品价值、玩法和安全信息；支持 B2C 消费者转化与 B2B 经销商合作；提供无需改代码即可维护产品、内容和经销商信息的运营后台；按角色隔离价格、订单和资料权限；支持搜索优化和行为分析；并具备响应式、多语言、权限审计、安全保护与可扩展数据结构。

4. 可以参考哪些需求文档模板？

徐熠潘 (E)



如果要为网站设计一份需求文档，有哪些模板可以参考？请联网检索。

GPT

不宜只套用一种模板，建议根据阶段组合使用：

模板	主要回答的问题	使用阶段
PRD（产品需求文档）	为什么做、为谁做、做什么、优先级如何	立项与团队对齐
网站功能需求说明	有哪些页面、用户在每页能做什么	网站功能规划
BRD（业务需求文档）	业务痛点、流程变化、成功标准	涉及经销商、订单、审核等业务协作时
SRS（软件需求规格说明书）	系统必须如何工作、异常与质量约束、如何验收	开发前定稿与课程交付

当前的《网站重构需求.docx》可作为全面的网站需求范例；最终选择的范围与规则宜用 SRS 固化为可实现、可测试、可验收的条目。

5. IEEE 风格的需求文档模板主要包含什么？

徐熠潘（E）

《requirements-doc-template.pdf》中主要有什么内容？

GPT

该文件是基于 IEEE 830-1998 改编的 SRS 模板，主线是把“为什么做、做什么、约束什么、如何验证”串起来。主体包括：

- 1. 引言：目的、读者、范围、术语和文档结构；
- 2. 总体描述：产品环境、主要功能、用户特征、外部接口、约束、假设与依赖；
- 3. 具体需求：按编号写出可实现、可检查的功能、质量和安全需求；
- 4. 建模：用例图、领域类图、数据字典、典型场景与时序图、状态图；
- 5. 原型：V1 可覆盖关键场景，V2 应能通过网页完成交互；
- 6. 参考资料与负责人联系方式。

对本项目最值得复用的是“具体需求 + 用例/场景 + 状态图 + 原型”的组合：自然语言说明范围，模型消除歧义，原型验证用户操作，最终测试验证需求。

6. SRS-Template 目录中的模板怎样使用？

徐熠潘（E）

SRS-Template 这份模板中有哪些内容？

GPT

该目录提供一套现代 Markdown 格式的 SRS 模板，以 IEEE 830 与 ISO/IEC/IEEE 29148 的结构为依据。其边界是“需求写清系统必须做

什么：实现方式交给架构设计和代码文档”。

主要文件为：

- `srs-template.md`: 带写作提示的完整 SRS 模板；
- `srs-template-bare.md`: 只保留章节骨架的简洁版；
- `req-template.md`: 单条需求的详细模板；
- `req-template-bare.md`: 单条需求的简洁版。

完整 SRS 通常涵盖引言、产品概述、接口与功能/质量/合规需求、验证方式及“需求 ID—测试证据”追踪矩阵，并可附数据字典、模型图、样例数据与变更记录。它的价值在于使每项需求都能追溯到验证证据，而非只列功能名称。

7. 从技术栈角度，如何保证网站的可维护性？

徐熠潘 (E)



从技术栈角度出发，如何设计网页才能够保证其可维护性？

GPT

可维护性的核心不是追逐热门框架，而是控制变化传播范围。可粗略理解为：

维护成本 $\propto$ 模块耦合度 $\times$ 需求变更频率

因此，应降低模块耦合，建立唯一数据来源，并使需求、接口、代码和测试能够追踪。针对“品牌官网 + B2C + 经销商门户 + 管理后台”这一业务复杂度，讨论中建议优先采用**模块化单体**，而非一开始拆为微服务：一个应用可部署、可调试，但内部按产品、订单、经销商、内容、身份等业务域划分模块，模块只公开必要能力。

讨论中的参考选型是：React + TypeScript 用于前台和后台，NestJS + TypeScript 按业务域组织后端，PostgreSQL 配合迁移脚本管理关系数据；图片、说明书等进入对象存储，耗时工作进入 Redis 队列。其可维护性意图是：共享组件与类型，隔离发布和权限边界，让数据库结构可复现，并避免慢任务阻塞交互。

需要注意：以上是原讨论针对目标网站规模给出的**参考方案**，并非对当前课程仓库的强制替换建议。当前共享实现已经约定 Vue 3 + TypeScript、Java 21 + Spring Boot 与 MySQL；若继续本仓库开发，应优先遵循现有技术约束，并将“模块化单体、领域边界、迁移、唯一数据源、接口与测试追踪”等可维护性原则迁移到既定栈中。

三、讨论形成的可执行共识

围绕后续需求分析、设计、实现与验证，讨论最终形成以下可执行共识：

✔ 形成的正式结论

1. 先明确目标用户、使用目标、项目范围和核心业务闭环，再展开页面设计与技术方案，避免在业务边界尚未确定时提前堆叠实现细节。
2. 以网站地图、用户流程、ER 图和角色权限矩阵构成最小设计集；当业务流程或状态复杂度提高时，再补充 BPMN、状态图、决策表和时序图，使设计文档与实际协作需要相匹配。
3. 将最终确认的功能、模块边界、异常处理、质量约束和验收方式写入 SRS，并建立需求 ID 到接口、代码、测试及验收证据的追踪关系，保证变更能够被定位和复核。
4. 前后端以 OpenAPI 等可执行接口契约协作，明确请求、响应、错误和兼容性规则；接口发生变化时同步更新实现与测试，防止契约与代码逐步偏离。
5. 项目以本地演示为目标时，控制与交付目标无关的生产运维复杂度，但保留权限控制、输入校验、错误处理、启动说明和核心测试，确保系统能够被稳定演示和复现。
6. 在既定课程技术栈内，以业务域模块化、共享类型与组件、数据库迁移和自动化测试降低后期维护成本，并将这些约束落实到目录、接口、实现和验证材料中。